



Chapter 3 – Agile Software Development

Topics covered



- ❑ Agile Manifesto

- ❑ Agile methods

- ❑ Agile user story

- ❑ Scaling Scrum

History



- ☐ Dissatisfaction in software industry 1980-1990
- ☐ Developed late 1990 to overcome problems in old methods
- ☐ Refactoring
- ☐ Pair programming
- ☐ Scrum
- ☐ XP
- ☐ CI
- ☐ Agile manifesto 2001
- ☐ TDD
- ☐ BDD
- ☐ Clean Code

Agile in a nutshell



- ☐ Rapid development and delivery
- ☐ Frequent delivery
- ☐ Incremental versions
- ☐ Minimal documentation
- ☐ Extensive tool support
- ☐ Interleaved activities
- ☐ Respond to changes quickly
- ☐ Reduced process overhead



Agile methods

The Agile Value System



Agile manifesto & Values



- *We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*
 1. *Individuals and interactions over processes and tools*
 2. *Working software over comprehensive documentation*
 3. *Customer collaboration over contract negotiation*
 4. *Responding to change over following a plan*
- *That is, while there is value in the items on the right, we value the items on the left more.*

The 12 principles of agile methods



1. Customer satisfaction by early and continuous delivery of valuable software.
2. Welcome changing requirements, even in late development.
3. Deliver working software frequently (weeks rather than months).
4. Close, daily cooperation between business people and developers
5. Projects are built around motivated individuals, who should be trusted
6. Face-to-face conversation is the best form of communication (co-location)

The 12 principles of agile methods



7. Working software is the primary measure of progress
8. Sustainable development, able to maintain a constant pace
9. Continuous attention to technical excellence and good design
10. Simplicity—the art of maximizing the amount of work not done—is essential
11. Best architectures, requirements, and designs emerge from self-organizing teams
12. Regularly, the team reflects on how to become more effective, and adjusts accordingly

Agile principles in a nutshell



Principle	Description
Customer involvement	<u>Customers</u> should be closely involved throughout the development process. Their role is <u>provide</u> and <u>prioritize</u> new system requirements and to <u>evaluate</u> the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without <u>prescriptive processes</u> .
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

When Agile is applicable



- ✧ Small/medium size products.
- ✧ Customer is available and involved
- ✧ Cross functional team
- ✧ Organization mindset

***“Agile is a mindset defined by values
guided by principles and manifested through many
different practices”***

– Ahmed Sidky, ICAgile Executive Director

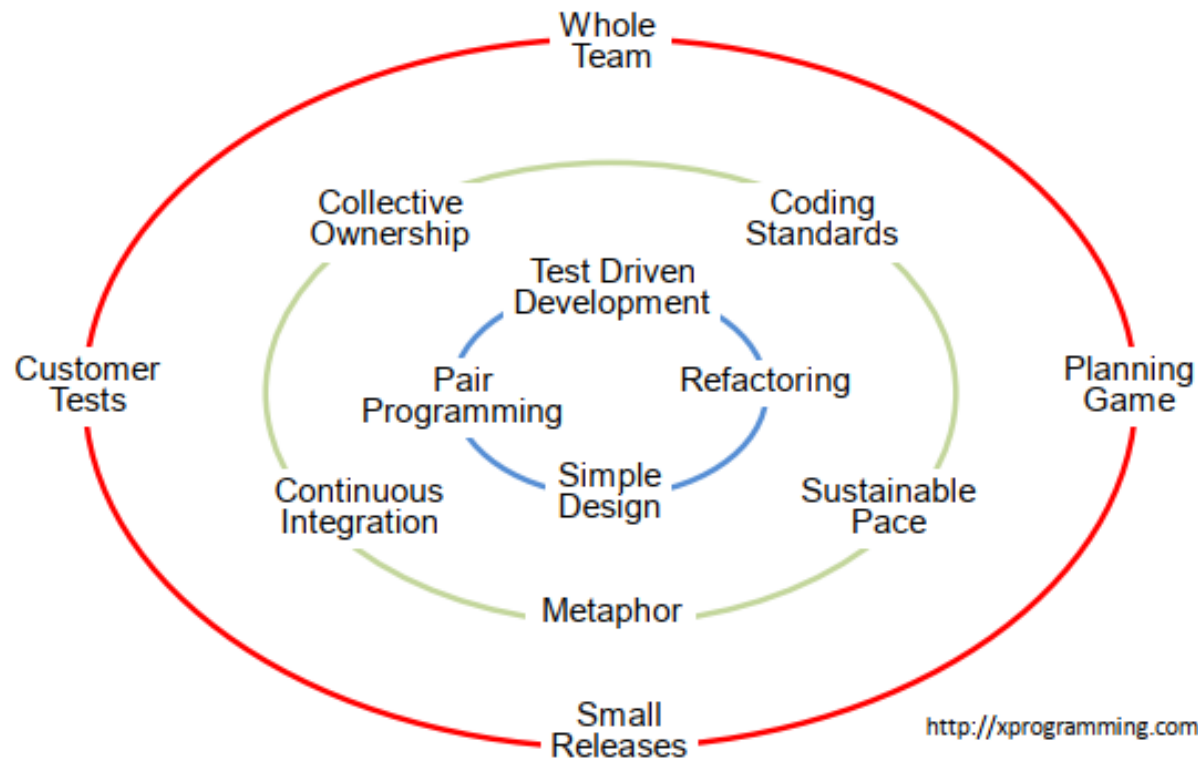


Agile development techniques

eXtreme Programming



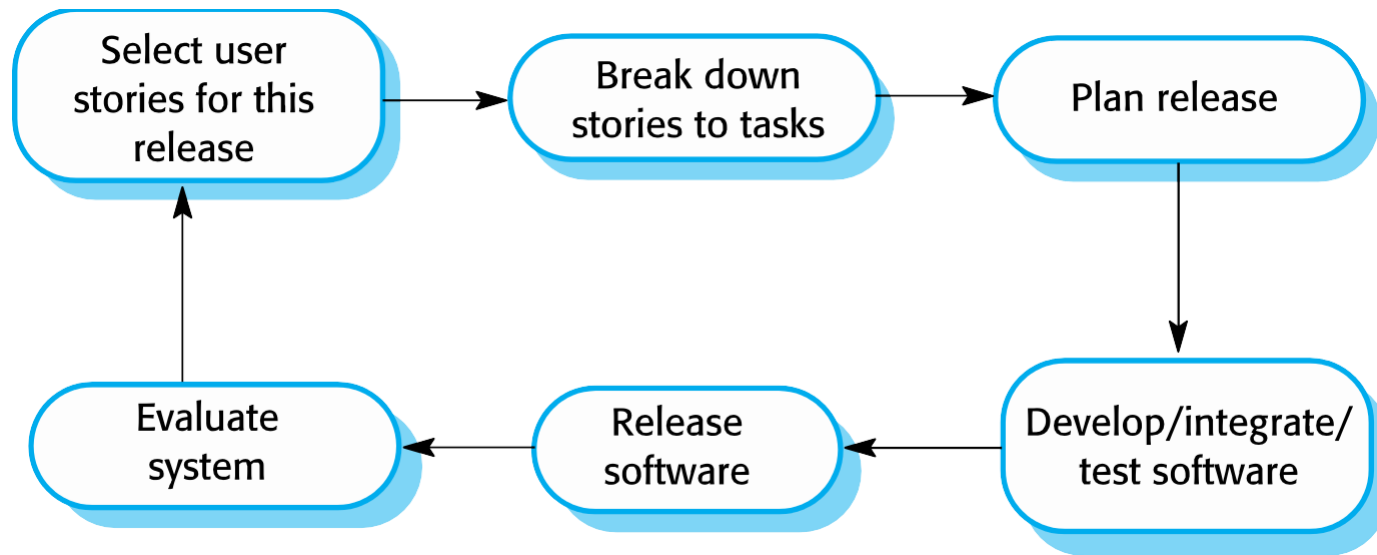
XP Practices



© Copyright Software Engineering Competence Center 2013

Introduction to Agile Software Development 34

The extreme programming release cycle



Extreme programming practices (a)



Principle or practice	Description
Incremental planning	<u>Requirements</u> are recorded on <u>story cards</u> and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'. See Figures 3.5 and 3.6.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Extreme programming practices (b)



Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of overtime are not considered acceptable as <u>the net effect is often to reduce code quality and medium term productivity</u>
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

How to map XP practices to Agile?



- ✧ Incremental development is supported through small, frequent system releases.
- ✧ Customer involvement means full-time customer engagement with the team.
- ✧ People not process through pair programming, collective ownership and a process that avoids long working hours.
- ✧ Change supported through regular system releases.
- ✧ Maintaining simplicity through constant refactoring of code.

Influential XP practices



- ✧ Difficult to integrate with management strategy in most organizations.
- ✧ Used practices
 - User stories for specification
 - Refactoring
 - Test-first development
 - Pair programming

User stories for requirements



User Stories are...

Cards:

- Stories are traditionally written on note cards
- Maybe annotated with notes, estimates, etc

Conversation:

- Stories are invitation to conversations → The details of Stories come out during conversations (with PO, Customers, Users, etc)

Confirmation:

- Acceptance Tests / Conditions of Satisfaction confirm that the story has been developed correctly

Source: XP Magazine, Ron Jeffries

© Copyright Software Engineering Competence Center 2013

Introduction to Agile Software Development 61

User story template



As a <User Role>,
I want <Goal/Desire>
so that <Reason/Benefit>

Sample user story



☰ Description

[Edit](#)

As a user I should have option to print any item with all the details, comments and other things. I should get printable view in browser and then should have option to print into different formats

Acceptance Criteria

- All the item information should be visible including title, ID, description, comments, attachment names, linked items tasks/issues/epics etc, associated items, dependencies etc
- On the preview page, I should option to print, download as
 - PDF
 - Word
 - XML(optional)
 - any other?
- All the items type should be printable
 - User Story
 - Epic
 - In case of Epic, we will show the related user stories and show only, ID, Title, Responsible, Status and priority
 - Tasks
 - Issues
- It should be possible to print the item details from almost anywhere, e.g.,
 - From boards widgets under context menu
 - In case of Epic or user story, from Epic or Backlog item context menu
 - From item detail view
 - From pop-up under context menu

Sample user story



Stories for a School Website

As a user, I want to manage my homework assignments

As a Teacher, I want to grade the assignments submitted by my students for each of my classes

As a parent, I want to instantly view my child's exam results for a specific class instead of waiting for the final consolidated report

As a Student, I want to be notified as soon as a new homework is assigned, so that I would be able to work instantly on my assignments

Sample user story



As a **teacher**, I want to manage my assignments

- I want to upload a new assignment for one of my classes
- I want view my classes assignments ordered by class and number of submissions
- I want to download a submitted assignment for grading

As a **Student**, I want to manage my assignments

- I want to view my assignments per class
- I want to download an assignment for one of my classes
- I want to upload a finished assignment

User story details



Details of a user story

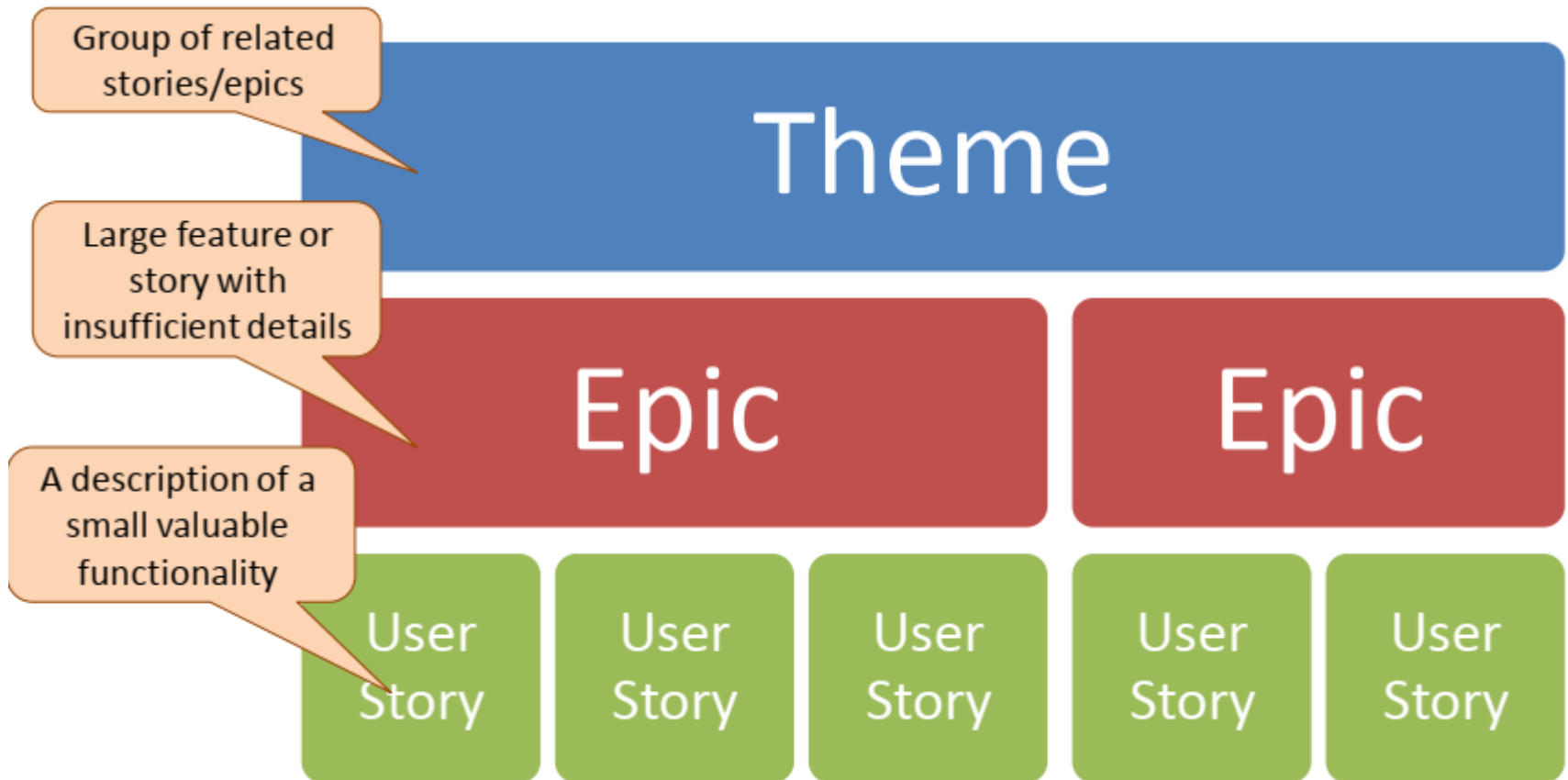
As a Student, I want to check my classes schedule

Through
Conversation

Details:

- I want my class schedule to show my classes across the days of the week
- I want my favorite classes to be highlighted
- I want my extra-curricular activities to be displayed in the table
- I want different color formats for my classes

Levels of requirements



Done definition



Definition of Done (DoD)

The below examples might be included in a User Story DoD:

- Unit tests passed
- Code reviewed
- Acceptance criteria met
- Functional Tests passed
- Non-Functional requirements met
- Product Owner accepts the User Story

Sample test case description



Test 4: Dose checking

Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

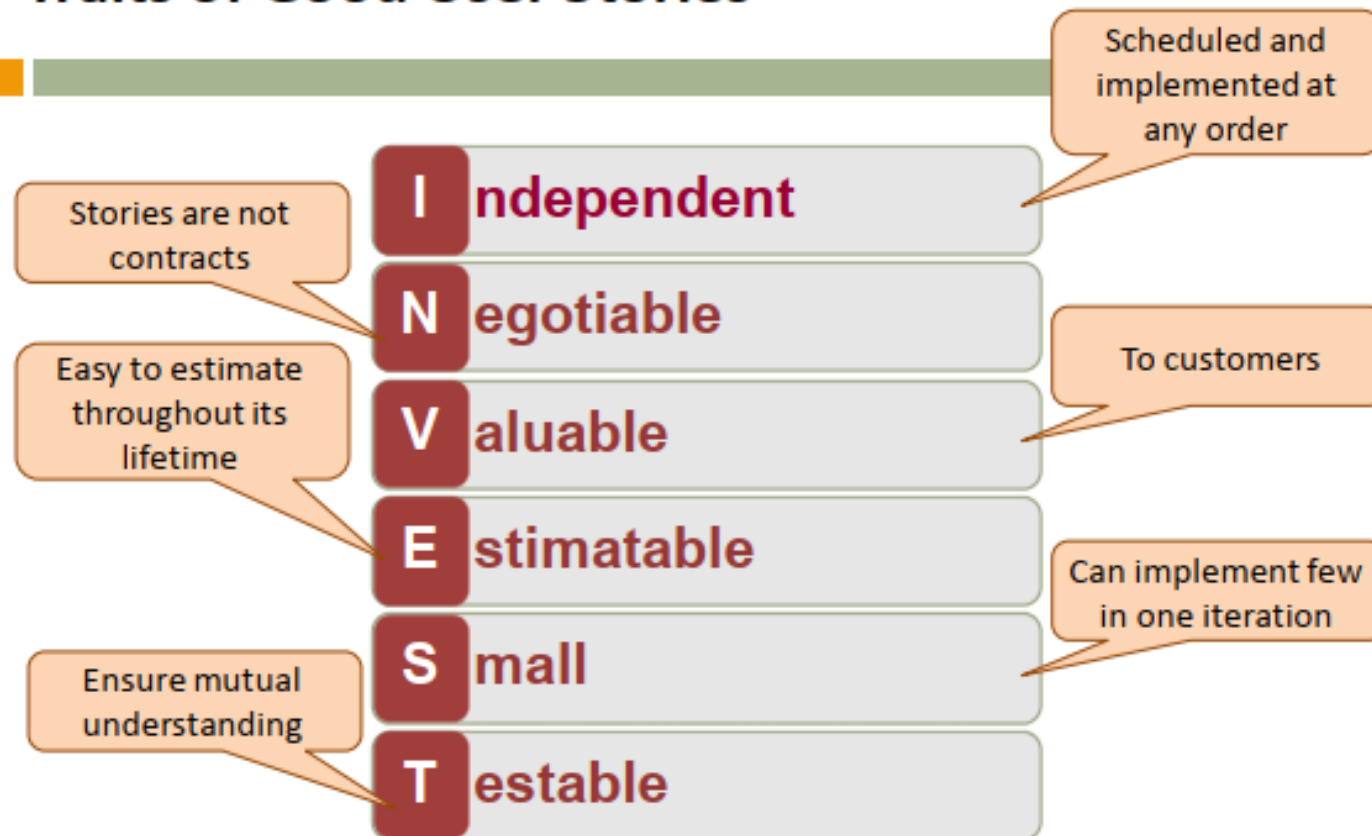
Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

Traits of Good User Stories



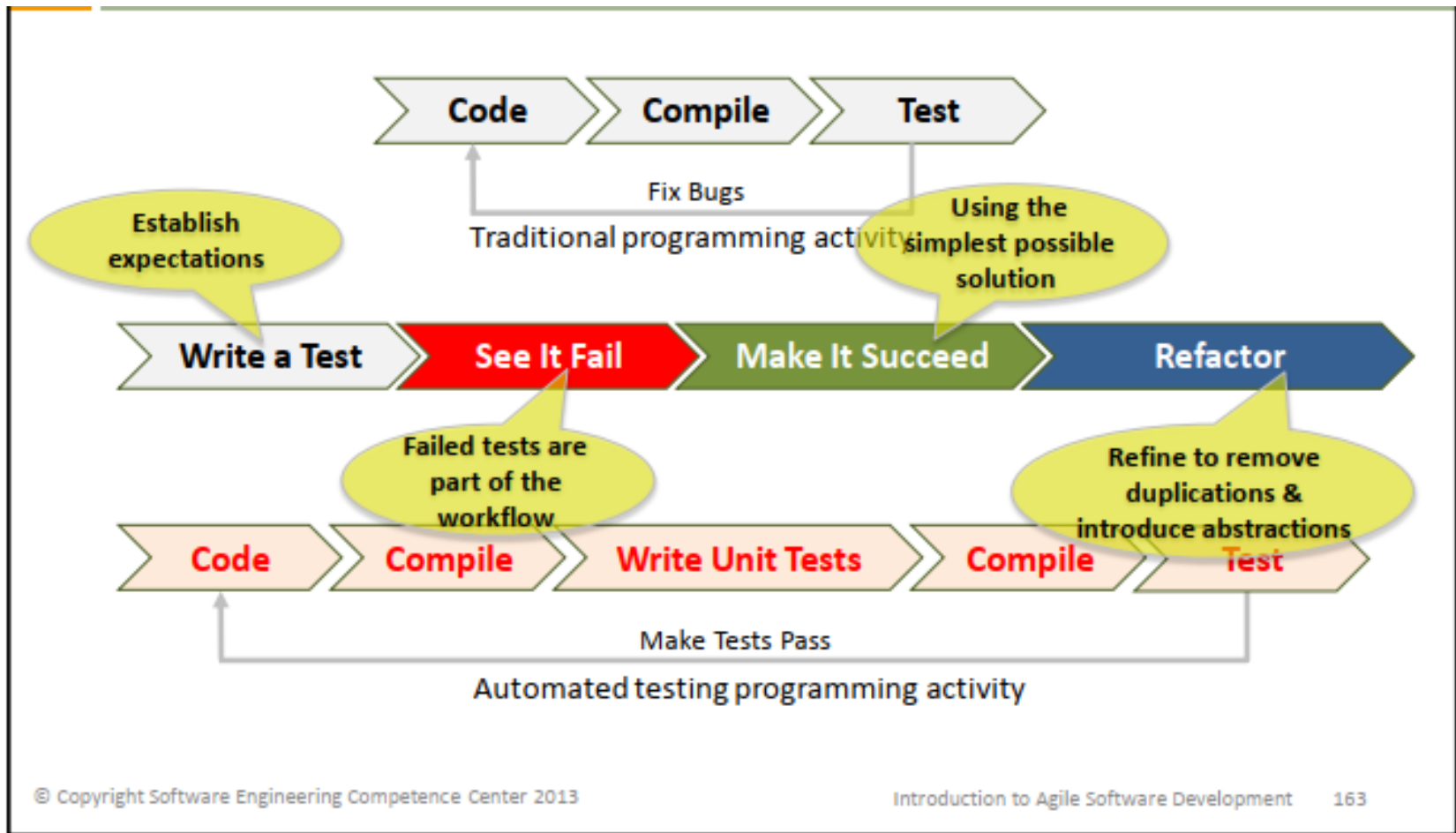
Source: Bill Wake (2003)

Refactoring



- ☐ Remove duplicate code.
- ☐ Logical naming for attributes and methods.
- ☐ Replace inline with method calls.
- ☐ Enhancing performance
- ☐ Documenting complex functions
- ☐ Re-architecture

TDD: Test Driven Development

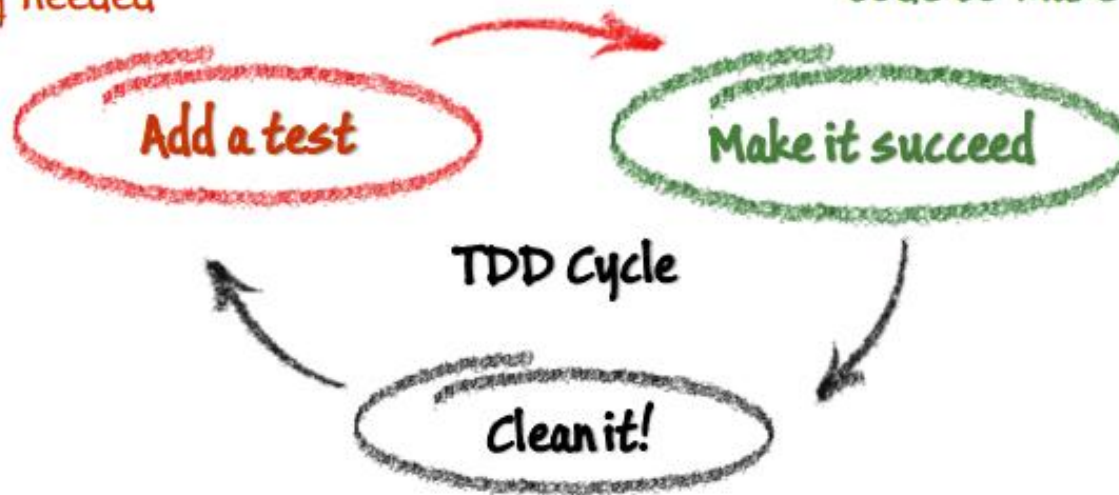


Practice: Red-Green-Refactor



Add a test which is
really needed

Write minimal production
code to make it succeed



Clean it! Keep all code base
structured and well-designed

Pair programming



- ✧ Pairs: developing code together
- ✧ Collective ownership
- ✧ Considered as informal review process.
- ✧ It encourages refactoring as the whole team can benefit from improving the system code.
- ✧ Knowledge sharing



Agile project management

Manger role



- Manage the project
 - Delivery on time
 - Within the budget
 - What should be delivered
 - When should be delivered
 - Who will develop the deliverables
- Agile project management requires a different approach, which is adapted to incremental development and the practices used in agile methods.

Scrum



- A lightweight agile software development framework for managing **product** and application **development**
- **Phases**
 - Discovery: specs, plan & design
 - Implementation: series of sprint cycles
 - Closure: user manuals, project retrospective, lessons learnt

Scrum terminology (a)



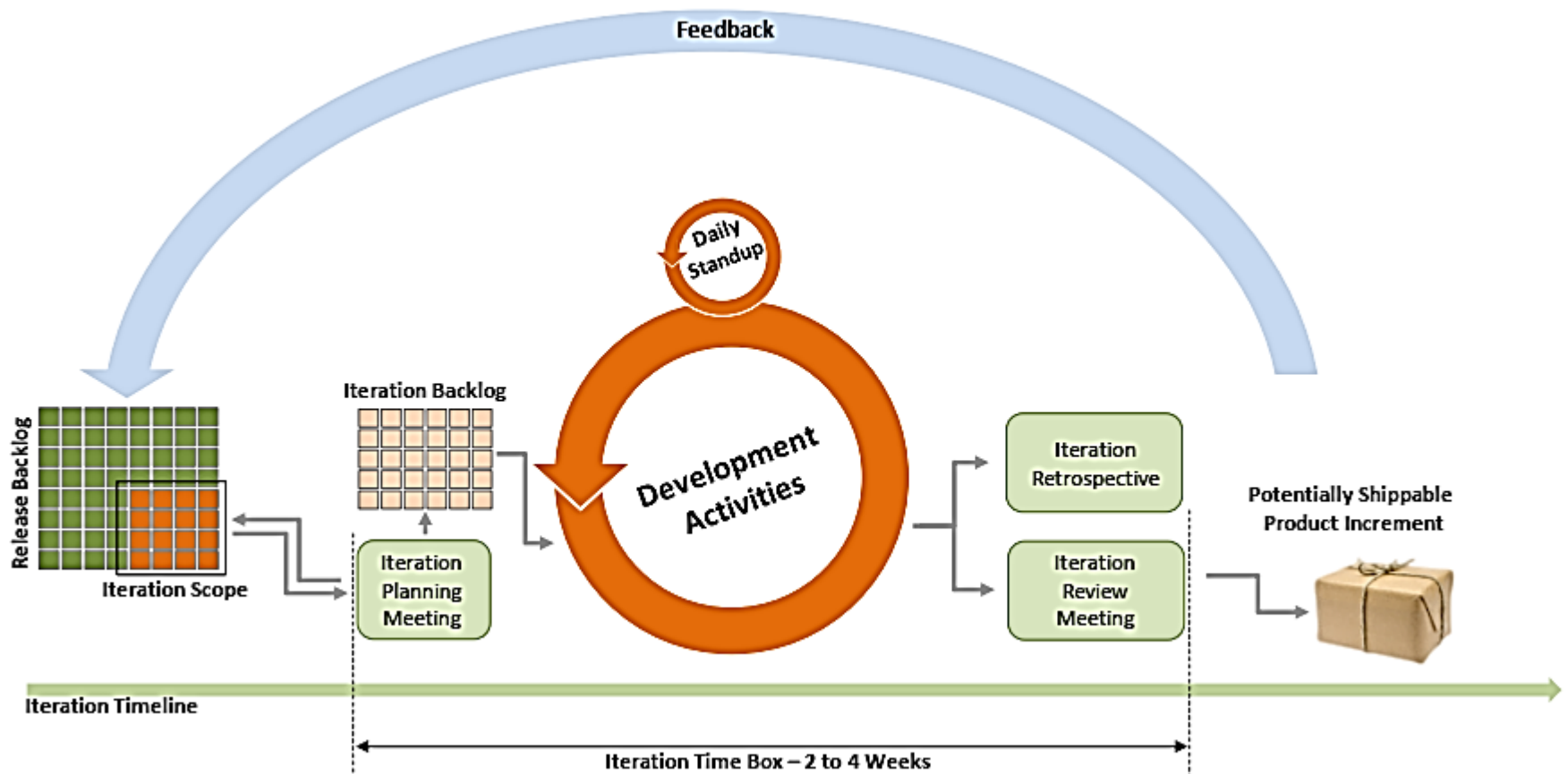
Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. <u>In practice, this is not always achievable.</u>
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs. <u>The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.</u>

Scrum terminology (b)

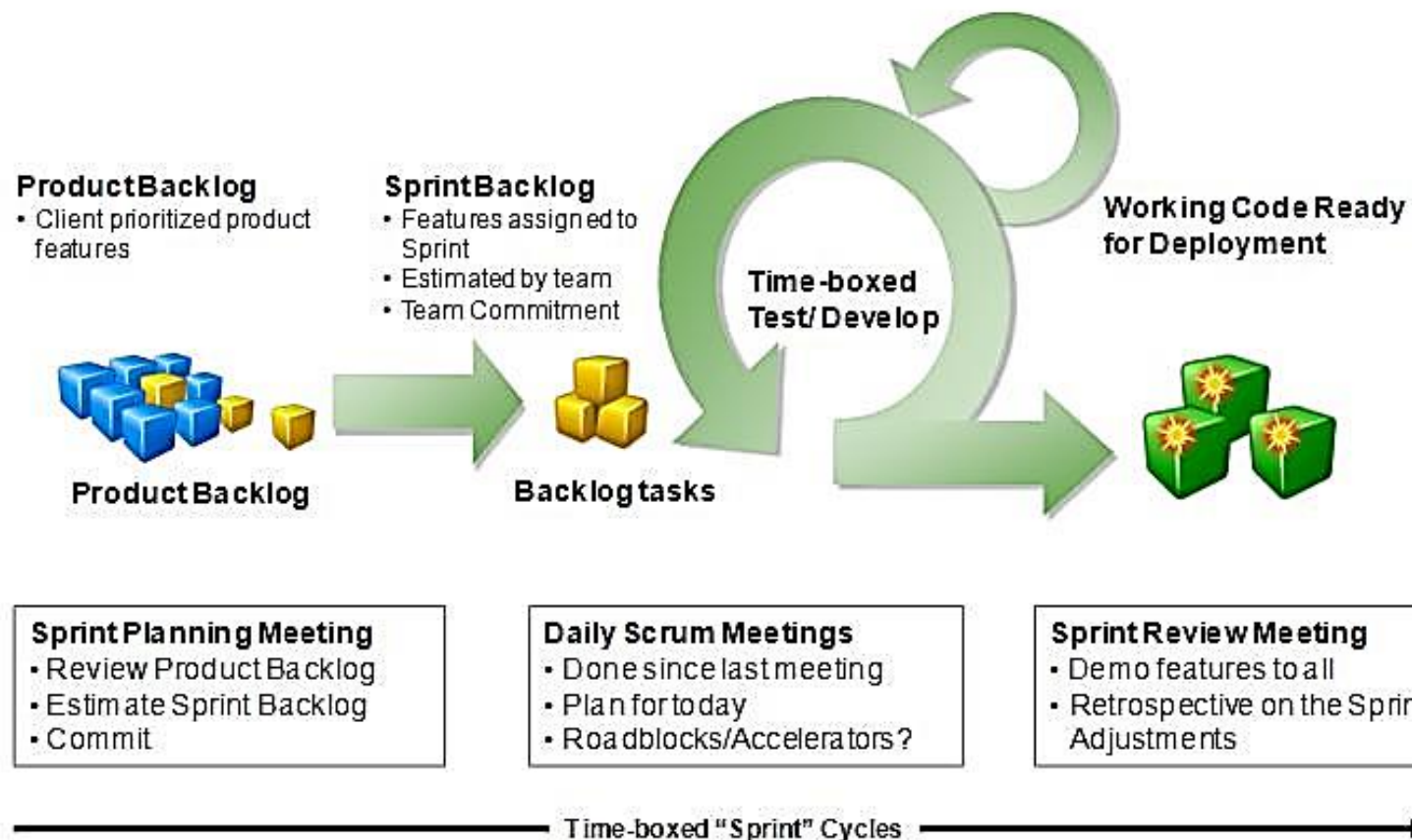


Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, <u>this should be a short face-to-face meeting that includes the whole team.</u>
ScrumMaster	The ScrumMaster is responsible for ensuring that the <u>Scrum process is followed and guides the team in the effective use of Scrum.</u> He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2-4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Scrum overview



The Scrum sprint cycle



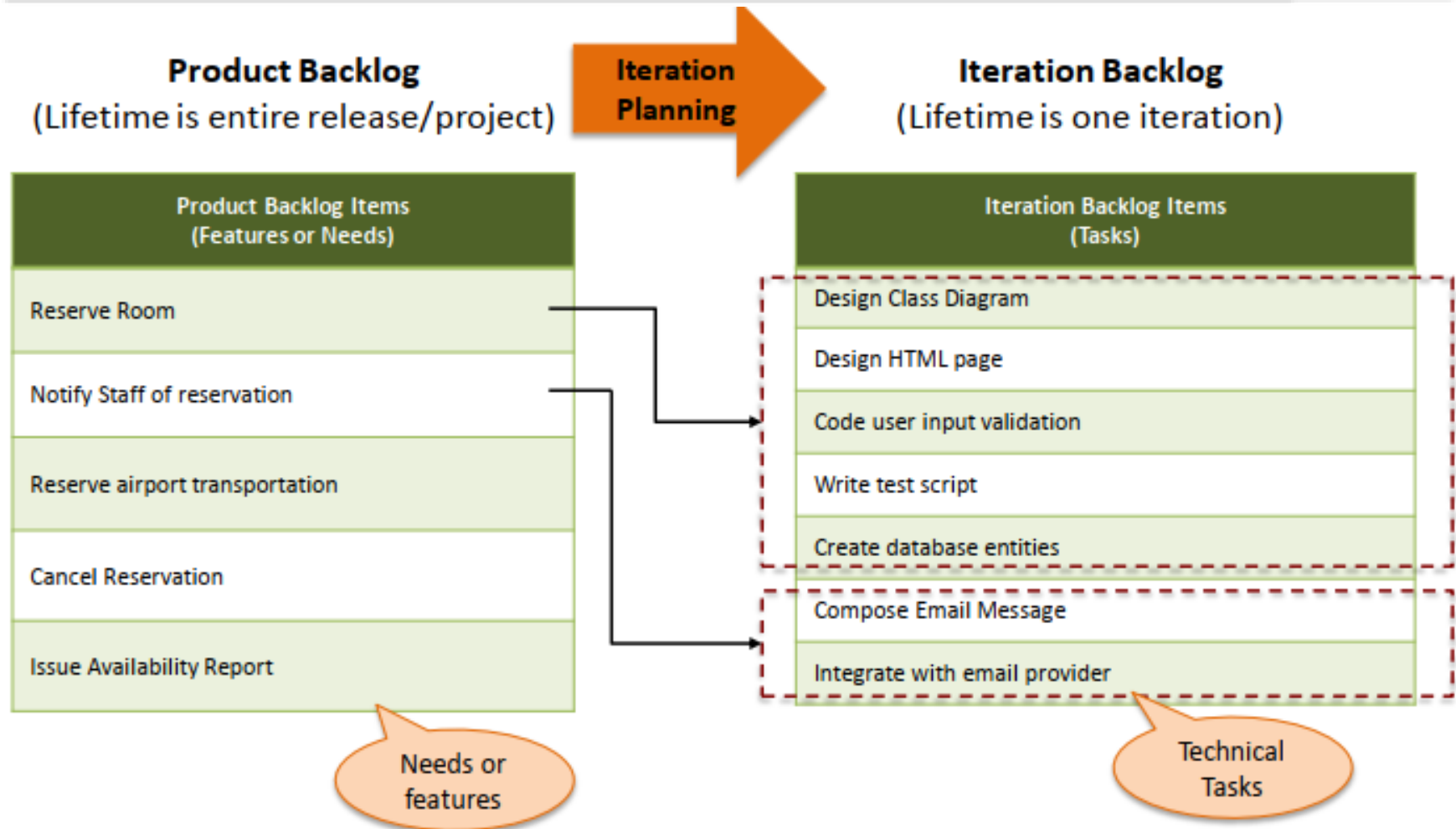
Product Backlog



- A listing of or **user needs** or **features**
- Each item should be valuable to the customers or users
- Items are associated with a priority and a size estimate

Backlog Item	Priority	Estimate
As a user, I want to reserve a room	100	8
As a user, I want to cancel a room reservation	100	3
As a user, I want to reserve airport transportation	60	5
As an employee, I want to be notified when a reservation is made	40	5

Product backlog vs iteration backlog



Release vs Sprint



Release

- Tested, packaged group of requirements
- Ready to go live to be used by end users
- Deliver value to the customer
- 3-6 months

Iteration/Sprint

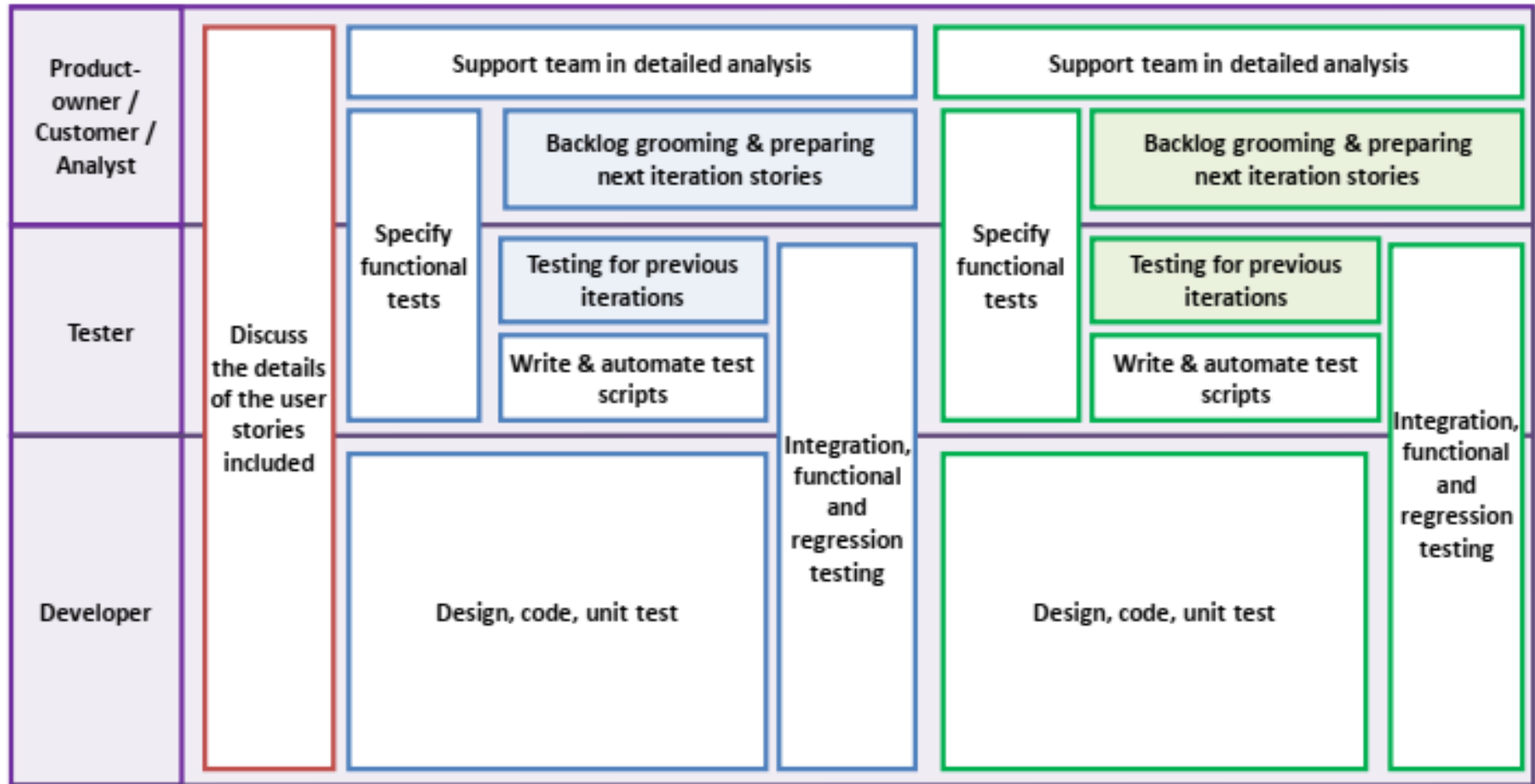
- Timeboxed development cycle within a release
- Outputs a “potentially shippable product increment”
- 1-4 weeks

The Sprint cycle



- ✧ Team committed to develop the selected stories
- ✧ Team is isolated from the customer
- ✧ External communication thru 'Scrum master'.
- ✧ At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Inside a sprint – little of everything all time



Scrum benefits



- ✧ The product is broken down into a set of manageable and understandable chunks.
- ✧ Unstable requirements do not hold up progress.
- ✧ The whole team have visibility of everything and consequently team communication is improved.
- ✧ Customers see on-time delivery and provide early feedback.
- ✧ Trust between customers and developers is established



User story estimation

Estimation



- Relative estimation is better than absolute
- Fibonacci numbers are mostly used
- No point of discussing one point difference
- The range has more lower value numbers
- Story point estimation

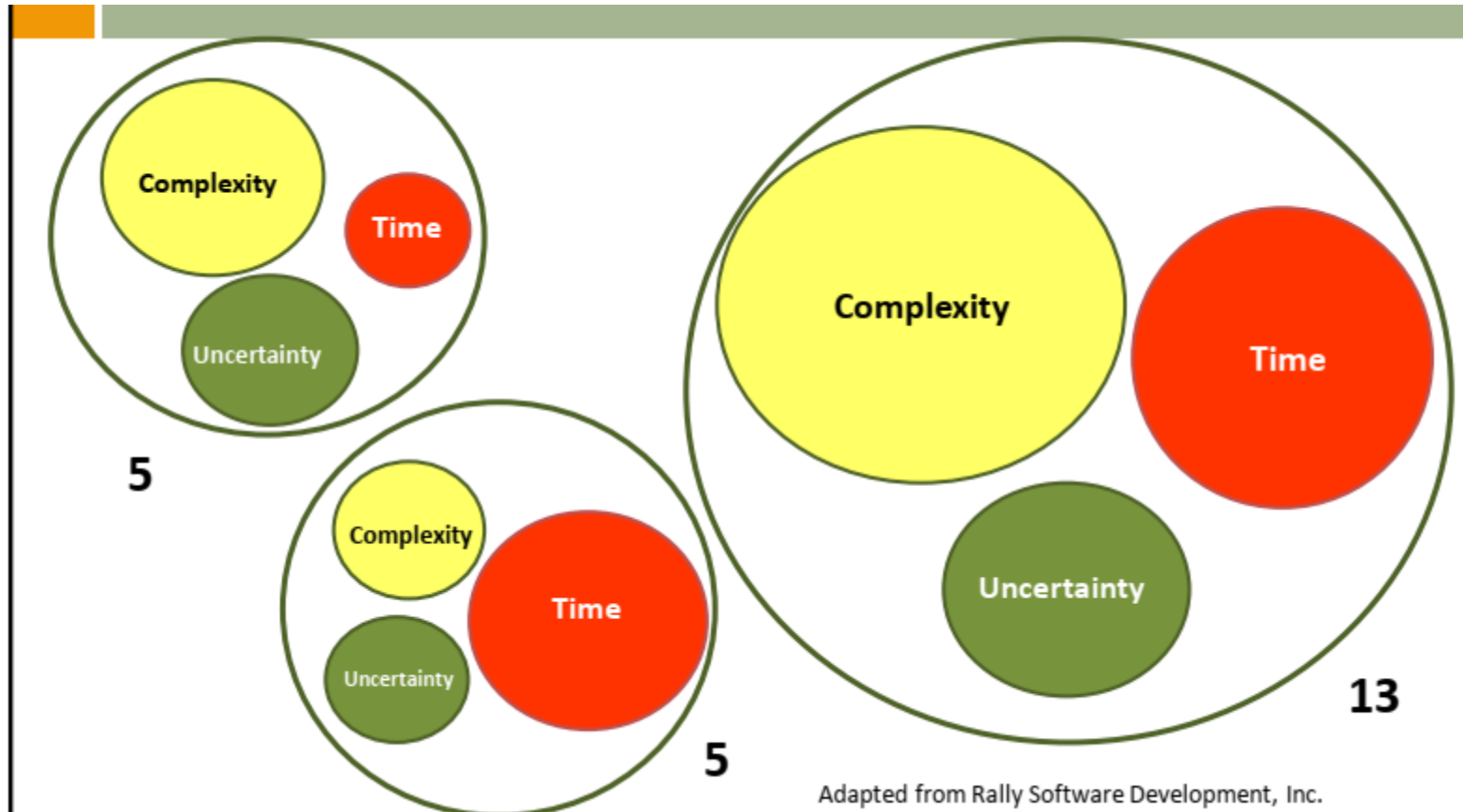


Story point



- Units of relative size
- Combination of a story's [Complexity/Difficulty –
Relative time – Uncertainty]
- Estimation don't expire
- Mapped to hours when assigned to developer

Story point



Planning Poker



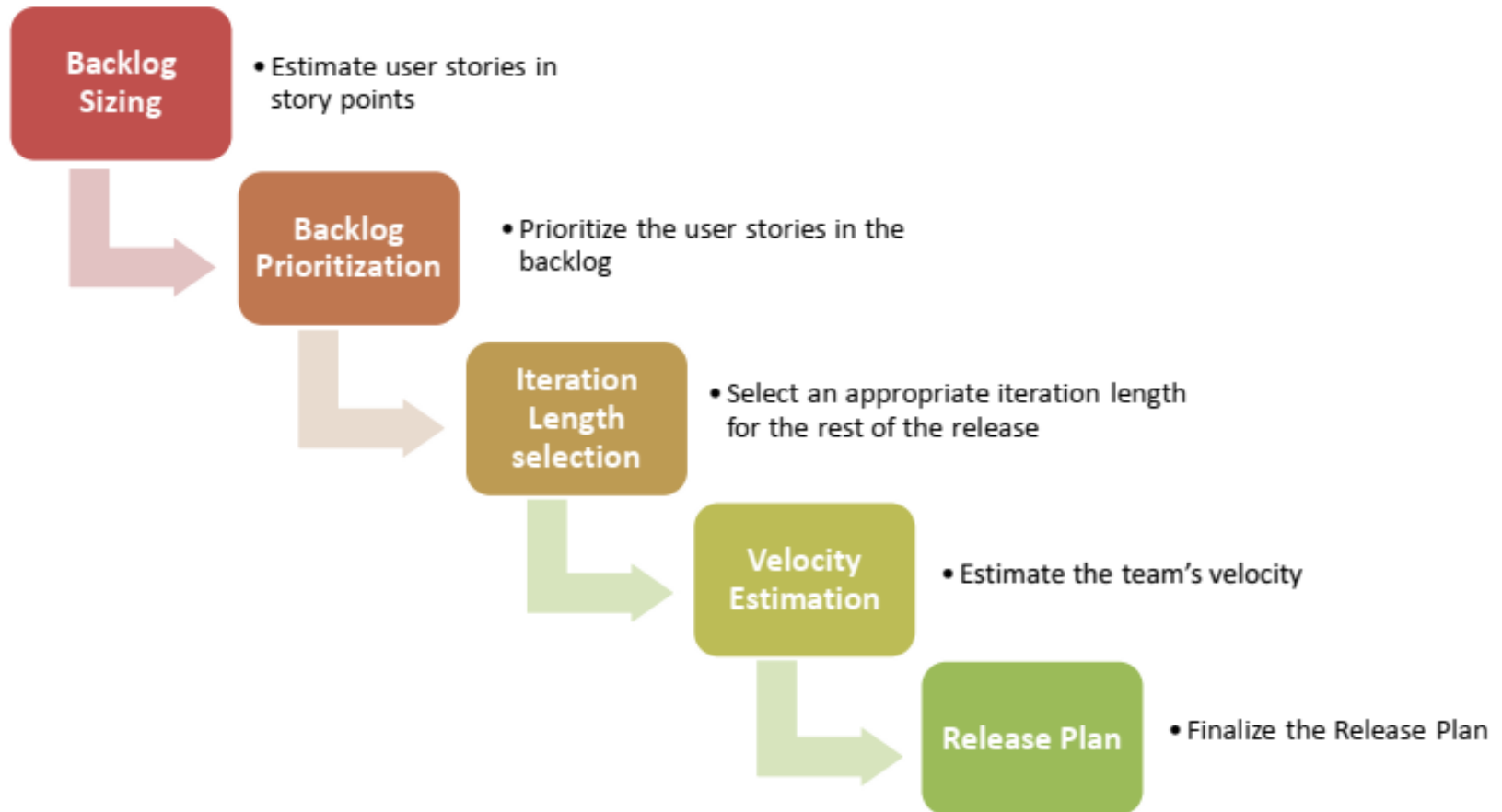
- For each user story
 - The Product Owner explains the story along with its conditions
 - Team member assigns Fibonacci number to the story using cards
 - All the estimates are revealed at the same time
 - If estimates are not equal then, the highest and lowest estimators should mainly discuss and share their reasons

Done Definition



- *Things* are either:
 - Not started: 0% finished
 - In Progress: 0% finished
 - Done: 100% finished!
- Avoids the 80% done syndrome!
- Teams establish their Definition of Done
- *Things* are “Done” **only** when they meet the DoD

Release planning



Velocity

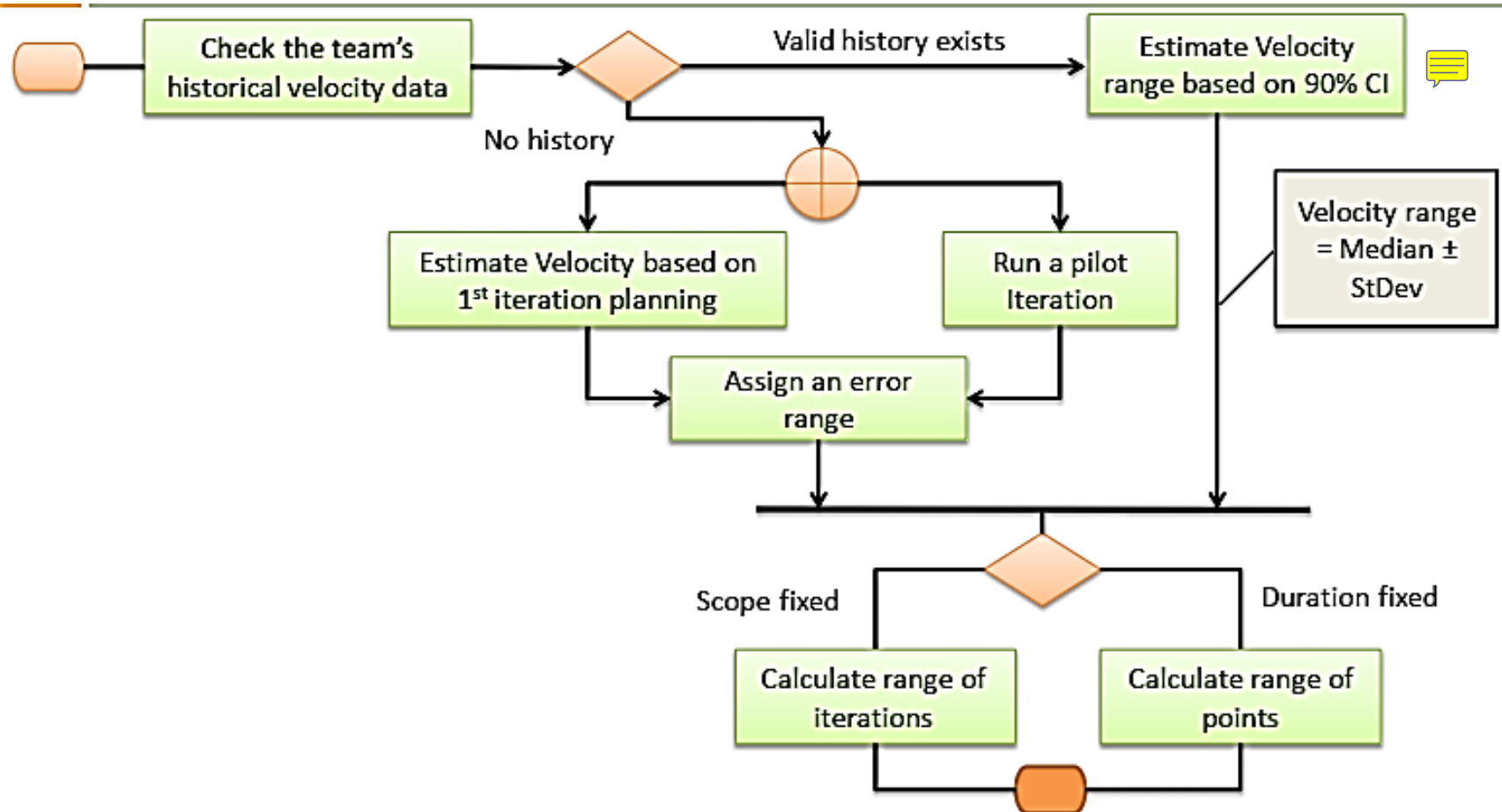


➤ Metric for measure of progress

$$\text{➤ } v = \frac{(\textit{number of story points})}{\textit{sprint}}$$

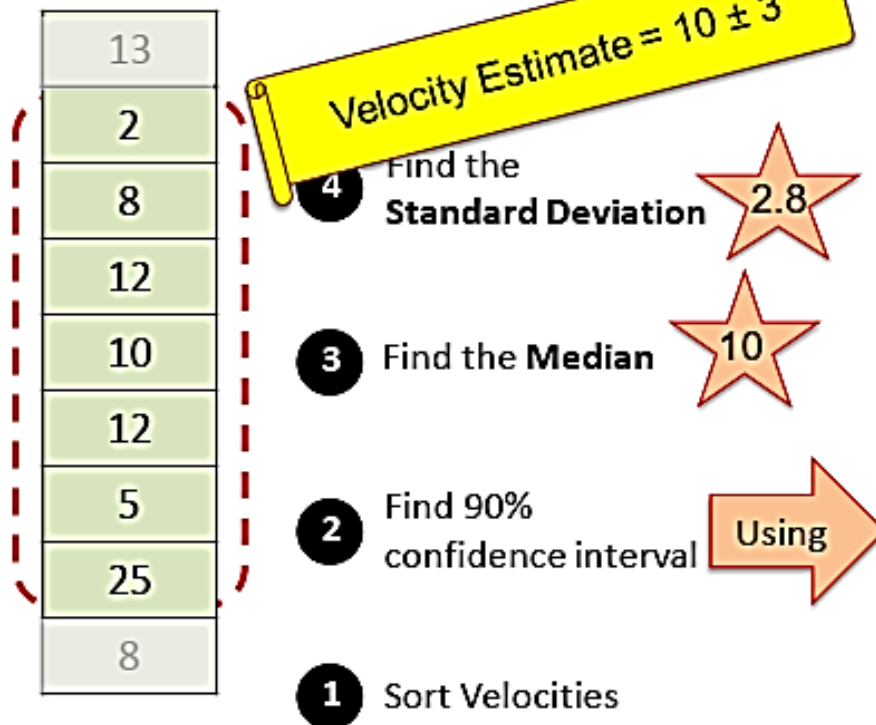
$$\text{➤ } \textit{project duration} = \frac{\textit{backlog size}}{\textit{velocity}}$$

Velocity estimation



Velocity estimation (Historical data)

Calculating velocity range from historical velocity data...



# of velocity values	Edge values to discard
5-7	0
8-10	1
11-12	2
13-15	3
16-17	4
18-20	5
21-22	6
23-25	7
26+	8

Adapted from Mike Cohn

Velocity estimation (Historical data)



B	C	D	E	F
Recorded velocity		Sorted velocity		90% CI
13		2		
2		5		5
8		8		8
12	1	8	2	8
10		10		10
12		12		12
5		12		12
25		13		13
8		25		
			Median	10
			SD	2.870208222

Velocity estimation (pilot sprint)



Option 1: Experiment !!!

- The team can run one or more pilot iteration(s)
- The actual velocity observed will be assumed as the average velocity for the rest of the release
- An error range will be formulated around the observed velocity value

Disadvantage of this approach:

- Customers have to wait till the team finishes one or two iterations to know the release plan
- The actual velocity of the iteration may not be reliable if something abnormal happens during the iteration

Velocity estimation (dry run)



Option 2: Dry run!

- The team uses the (Commitment Driven Planning) approach to estimate the velocity of the first iteration of the release
- The (Commitment Driven Planning) approach doesn't take in consideration the velocity of previous iterations or releases
- An error range will be formulated around the velocity value estimated

Considerations:

- The team should consider when using this approach that the iteration used to estimate the velocity is a normal one operating with the normal capacity and team setup

Velocity (Estimated – Planned - Actual)



- Assume the release planned velocity range is 7-13 points per iteration. Consider this table:

Iteration	Release Estimated Velocity	Iteration Planned Velocity	Iteration Actual velocity
1	10	8	9
2	10	11	5
3	10	9	14
4	10	10	12
...	...	...	...



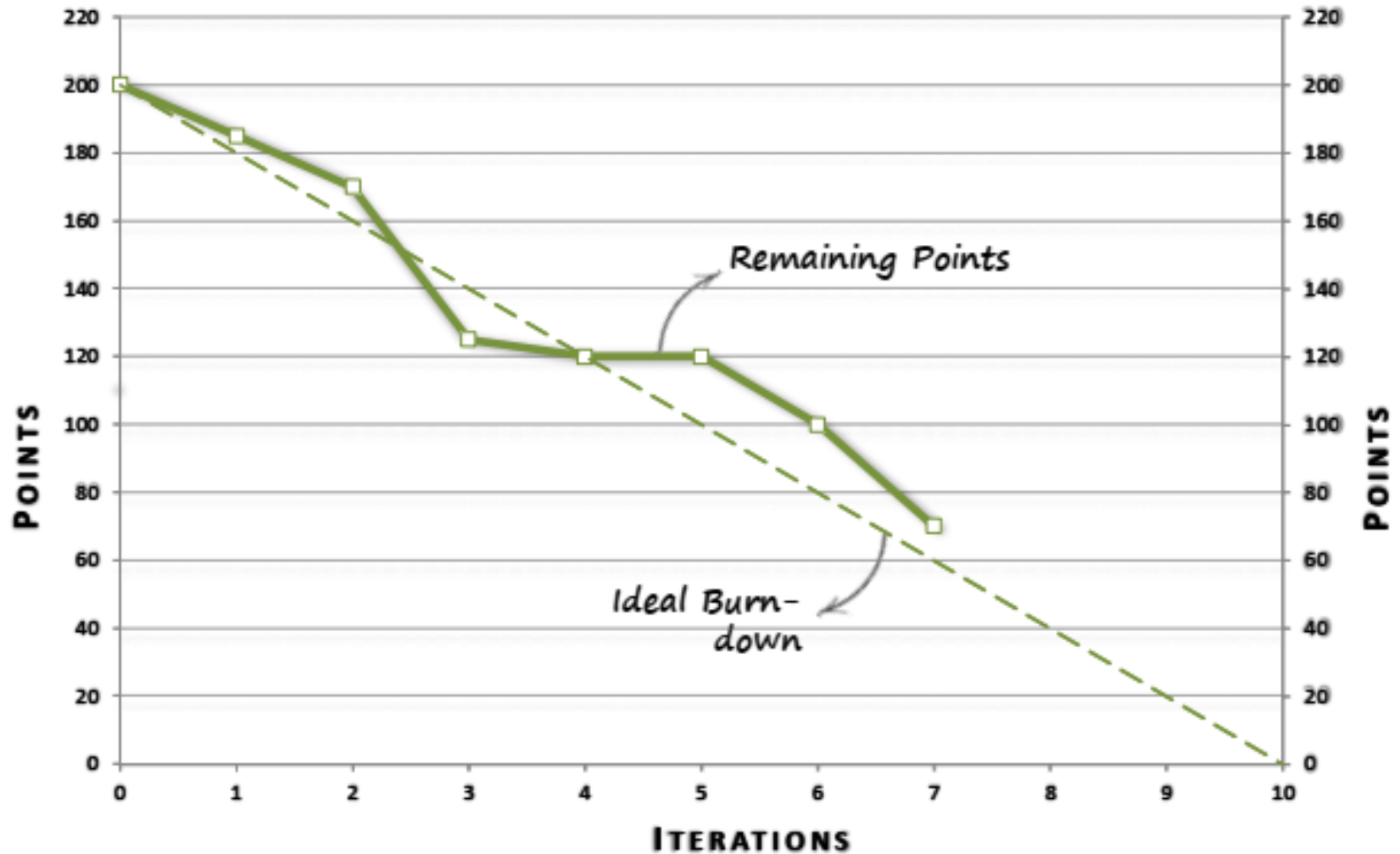
Agile project tracking & control

Tracking Agile project

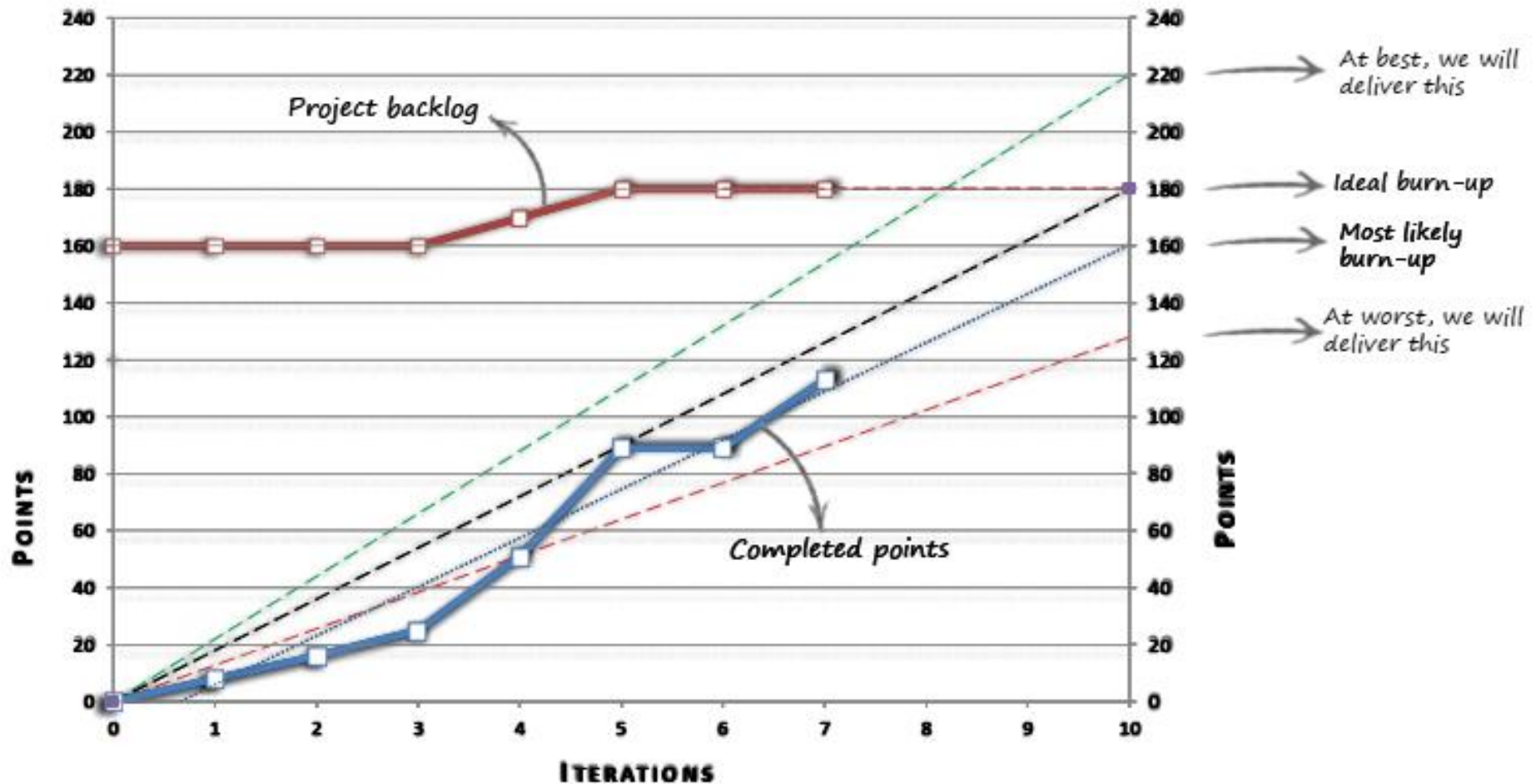


	Metrics	Indicators	Recurrence
Release Tracking	Backlog Points	Release Burn-down Release Burn-up	Every Iteration
	Team Velocity		
Iteration Tracking	Actual Effort	Iteration Burn-down Iteration Burn-up	Every Day
	Remaining Effort		
Daily Tracking	Number of Impediments	None	Every Day
	Your senses		

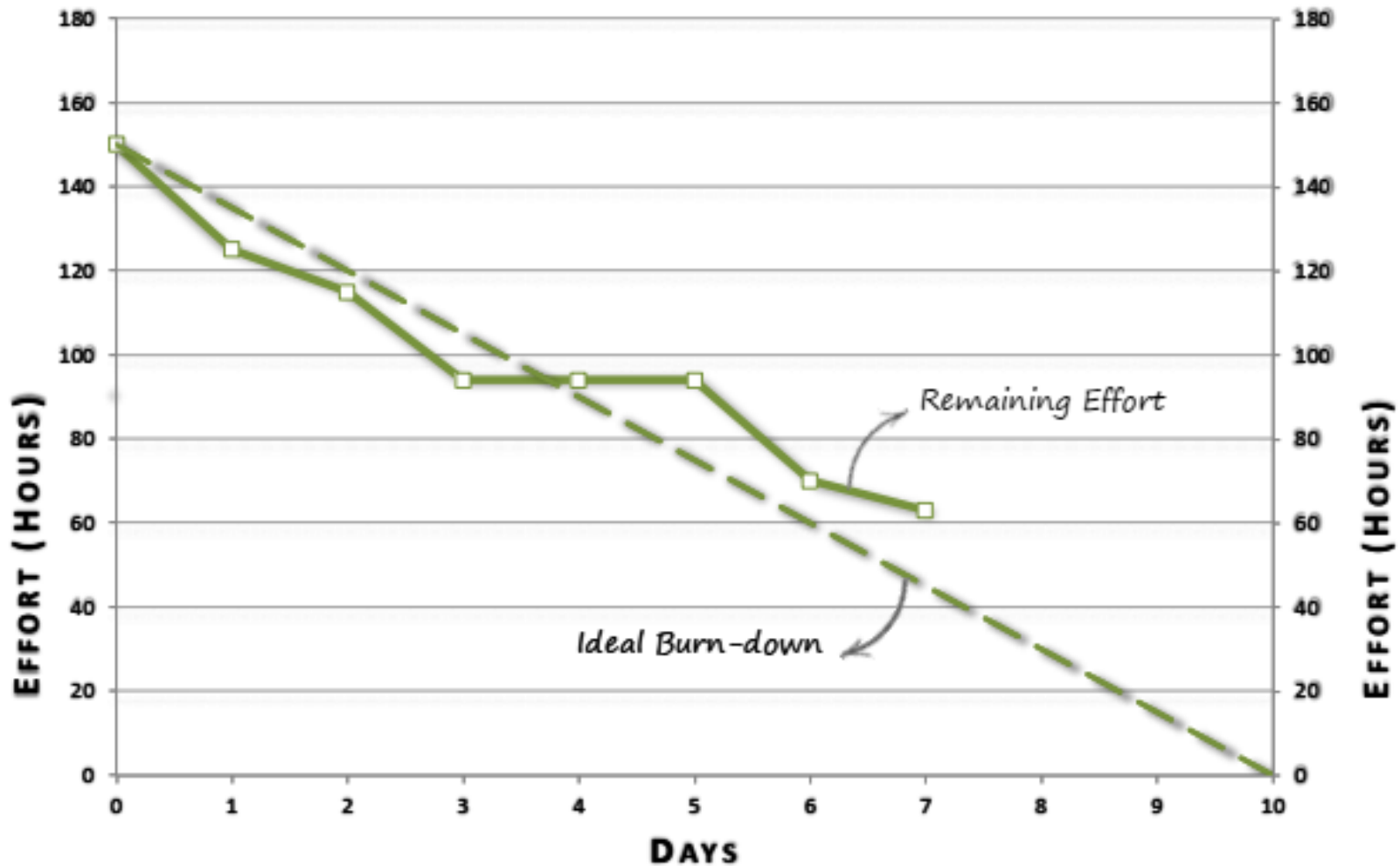
Release tracking: Burn-down chart



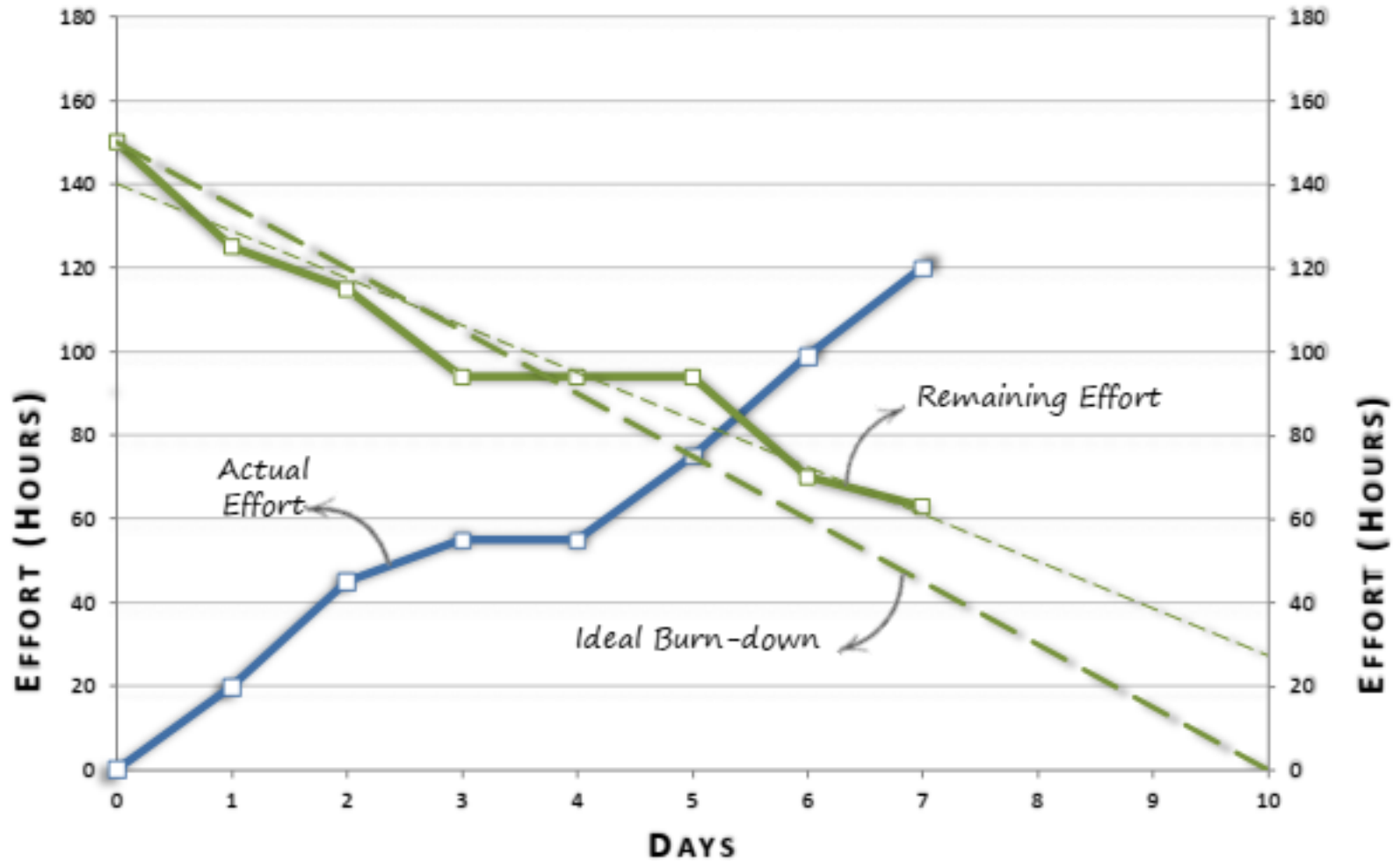
Release tracking: Burn-up chart



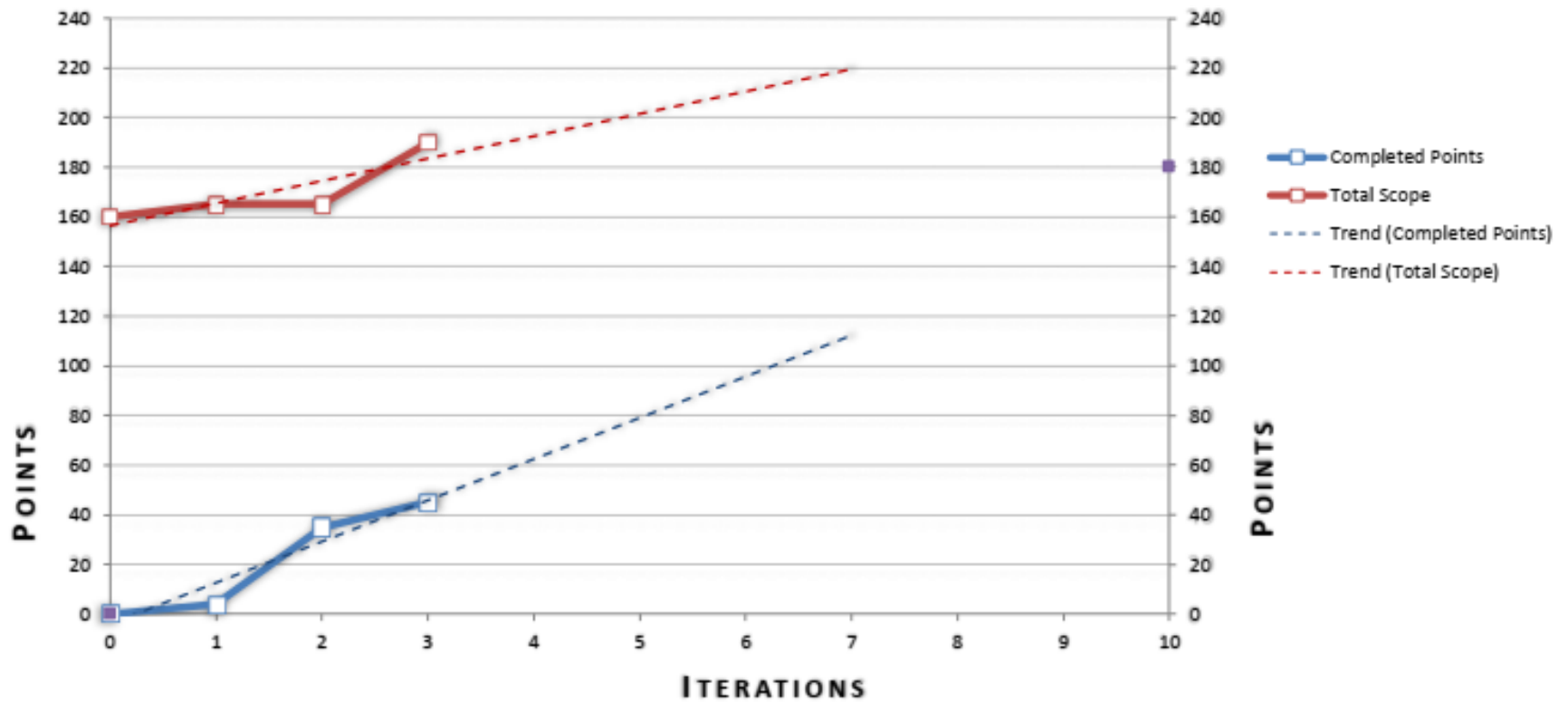
Iteration tracking: Burn-down chart



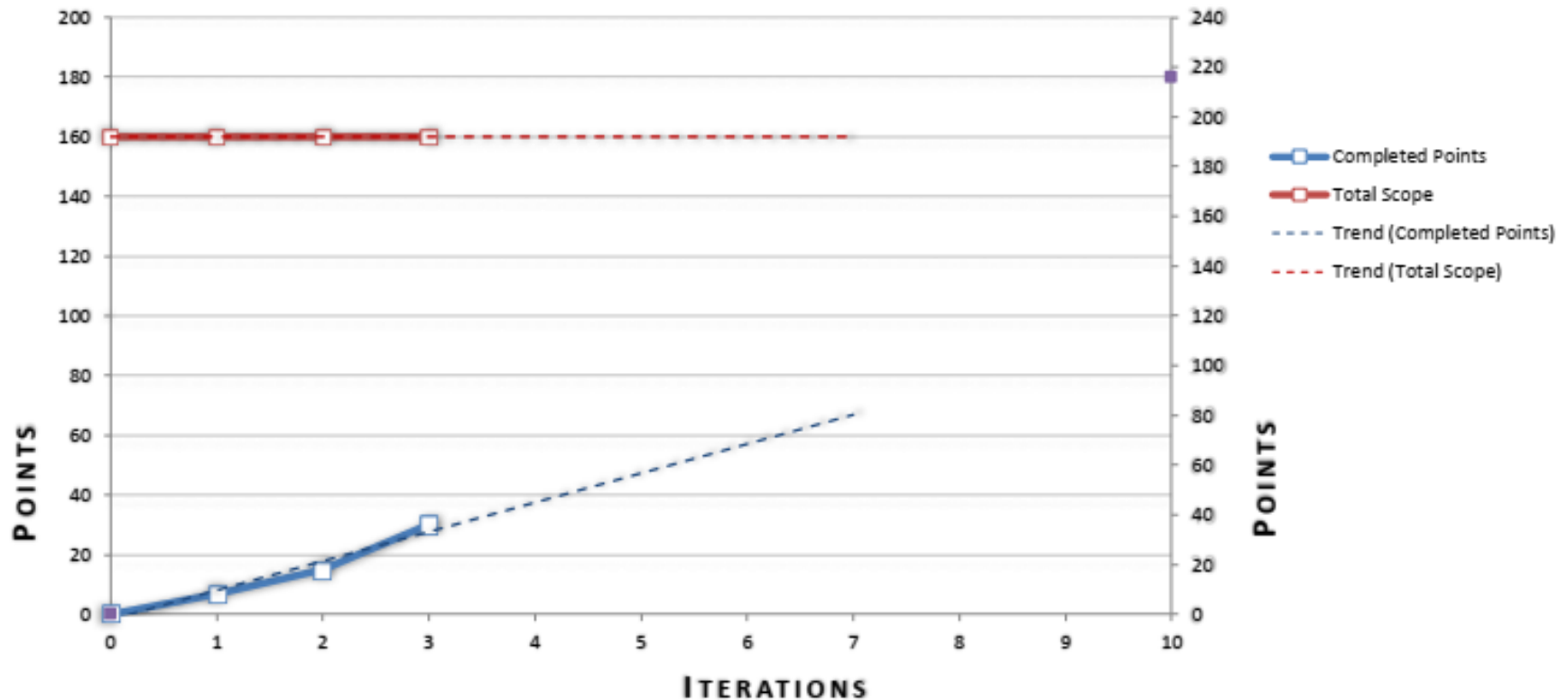
Iteration tracking: Burn-up chart



Would you take corrective action?



What about this one?



Conclusion



- ✧ Agile is **NOT** a silver bullet
- ✧ Agile is **NOT** a process, rather, methodologies and mind-set
- ✧ Agile implementations are **NOT** document-less
- ✧ Agile teams are **NOT** constantly under pressure
- ✧ An Agile environment is **NOT** chaotic and free of discipline

Abbreviation	Value
II	Individuals and interactions over processes and tools
WS	Working software over comprehensive documentation
CL	Customer collaboration over contract negotiation
RC	Responding to change over following a plan

Questions



1. What is the difference between agile values, principles and techniques?
2. based on your understanding, summarize the agile manifesto (values).
3. Agile philosophy is to welcome changes. Discuss!
4. Simplicity is essential. Discuss!
5. What is the difference between micro-managed and self-managed team? which is best?

Questions



6. Based on your understanding, discuss the following terms

Incremental planning

Test driven development

Customer involvement

Continuous integration

Pair programming

7. Mention some benefits of pair programming.

Questions



8. Mention the different level of requirements in Agile requirement specification activity
9. What is user story? what are the main components of user story?
10. What is the difference between user story and epic?
11. What is the Definition of Done? who is responsible to put such definition?
12. Mention some items that would be valid in the definition of done.

Questions



- 13. a good user story is the one that is independent, valuable, estimatable and negotiable. Discuss!
- 14. What is the difference between TDD and Code-first development?
- 15. What is Agile Scrum? what is it used for?
- 16. What is the role product owner and scrum master in Agile team?
- 17. What is the difference between sprint & release?
- 16. What is meant by: product backlog, velocity, potentially shippable product?

Questions



17. What is the measurement metric for user stories?

18. How do you estimate the velocity of the team for a new project?

mention the methods used of quantifying team velocity.

19. What is burn-down chart? what is it used for?